



BLOODBUDDY: A WEB-BASED BLOOD DONOR MANAGEMENT SYSTEM FOR NEPAL

**FINAL PROJECT REPORT
CPJ119 ADVANCED JAVA PROGRAMMING**

By:

SAKSHAM MAHARJAN (C30101250024)

BACHELOR OF INFORMATION AND COMMUNICATION TECHNOLOGY (BICT)

SCHOOL OF SCIENCE AND TECHNOLOGY

ASIA e UNIVERSITY

Supervisor: Mr. Binay Malla

DATE OF SUBMISSION: 15th Sep, 2026

DISCLAIMER

This report, titled "BloodBuddy: A Web-Based Blood Donor Management System for Nepal," has been prepared to fulfil the academic requirements of the Advanced Java Programming (CPJ119) module of the Bachelor of Information and Communication Technology (BICT) programme at Virinchi College, Asia e University. The system described herein was developed solely for educational and assessment purposes and is not intended for clinical or commercial deployment.

The five partner hospitals named in this report — Tribhuvan University Teaching Hospital, Bir Hospital, Patan Hospital, Manmohan Memorial Community Hospital, and Nepal Medical College Teaching Hospital — are named as the intended integration partners defined in the approved project proposal. All hospital, donor, request, and inventory data shown in the system are seeded demonstration data and do not represent live medical records, live blood bank stock, or any agreement with the named institutions. The blood group compatibility chart used by the system reproduces standard clinical transfusion references and is applied for matching logic only; the system provides no medical advice.

The demonstration build persists data in the browser for assessment purposes; the database-backed implementation is specified in Chapters 3 and 4. Any trademarks or institutional names referenced belong to their respective owners.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my academic supervisor, Mr. Binay Malla, for his invaluable guidance and constructive feedback throughout this project — from the initial proposal and problem analysis through to the design, implementation, and verification of BloodBuddy. His emphasis on grounding the system in the documented realities of blood request management in Nepal, and on enforcing role-based access rather than merely presenting it, shaped both the technical quality of this project and my own approach to software development.

I am grateful to the School of Science and Technology, Asia e University, and the faculty and staff of Virinchi College for providing the learning environment in which the theoretical foundations of this project — object-oriented design, database systems, web technologies, and software engineering practice — were developed. I also thank my lecturer, Mr. Asish Koirala, whose teaching in Advanced Java was instrumental in shaping the technical direction of this work.

I also thank my fellow students and peers for their constructive discussions, particularly during the requirement-analysis phase, and my family for their continued support and encouragement throughout my academic journey.

ABSTRACT

BloodBuddy is a web-based blood donor management system for Nepal, developed as the final project for the Advanced Java Programming (CPJ119) module. The system addresses the absence of any organised digital platform for blood donor discovery in Nepal, where urgent requests are currently circulated as informal social-media appeals with no guarantee of reaching compatible, available donors in time. BloodBuddy serves four user roles: donors register with their blood group and district and toggle their availability; requesters search compatible donors, submit blood requests, and track them through a status lifecycle; hospital staff manage blood bank inventory and fulfil incoming requests; and an administrative role moderates requests, manages users, and monitors platform analytics.

The system is designed around a strict N-Tier architecture (Controller → Service → Repository) using Spring MVC, with Hibernate/JPA mapping six related entities — USER, DONOR, BLOOD_REQUEST, HOSPITAL, BLOOD_INVENTORY, and EMAIL_NOTIFICATION — onto a MySQL database. Donor matching is driven by a blood group compatibility chart applied in the recipient-to-donor direction, and the design provides dedicated, hospital-scoped REST endpoints (/api/hospitals/{hospitalId}/...) for five partner hospitals so that each institution can integrate over its own data only. Role-based access control, validation on both the client and the server, database-stored images (BLOB/Base64), and SMTP notification with credentials supplied through environment variables complete the security and service design. The frontend is a fully responsive, framework-free HTML/CSS/JavaScript application that communicates with the backend exclusively through an AJAX/Fetch API facade mirroring the documented REST endpoints.

The completed frontend — 23 pages covering all four role portals, the public emergency flow, and every mandatory §2.1 capability of the assignment brief — was verified by 266 automated functional checks, a 92/92 responsive layout check across four viewport widths, and a 16/16 real-HTTP transport check against a REST stub, with server-side components scheduled for the implementation phase. The completed system demonstrates that a disciplined, well-structured Java web stack can deliver a secure and genuinely usable blood donor management platform tailored to the Nepali context.

Keywords: Spring MVC, Hibernate, JPA, MySQL, RBAC, REST API, Blood Bank, Donor Management, N-Tier Architecture, SMTP, AJAX/Fetch, Nepal

TABLE OF CONTENTS

DISCLAIMER	2
ACKNOWLEDGEMENTS.....	3
ABSTRACT	4
TABLE OF CONTENTS	5
LIST OF TABLES.....	7
LIST OF FIGURES.....	8
LIST OF ABBREVIATIONS	9
CHAPTER 1: INTRODUCTION.....	10
1.1 Overview.....	10
1.2 Problem Statement	10
1.3 Objectives.....	11
1.4 Scope	11
1.5 Justification and Significance of the Study	12
CHAPTER 2: LITERATURE REVIEW	13
2.1 Introduction.....	13
2.2 Related Concepts	13
2.2.1 Blood Donation Management	13
2.2.2 Web-Based Information Systems and Spring MVC	13
2.2.3 Hibernate ORM and Entity Relationships.....	13
2.2.4 RESTful APIs and AJAX/Fetch Front Ends.....	13
2.2.5 Web Security: Role-Based Access Control.....	13
2.2.6 Image Storage Strategies	14
2.3 Review of Existing Systems	14
2.3.1 Hamro Donor (Nepal)	14
2.3.2 Sankalp India (Comparative Reference)	14
2.3.3 The Manual Social-Media Workflow (Current Practice in Nepal).....	14
2.4 The Market Gap and BloodBuddy's Position	15
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN	16
3.1 Requirements Summary	16
3.2 System Architecture	16
3.3 Use Case Model	18
3.4 Entity Relationship Design	18
3.5 Class Design.....	19
3.6 Data Flow	20
3.7 Business Rules	20

3.8 Design Refinements from Proposal to Implementation.....	21
3.9 Security Design.....	22
CHAPTER 4: IMPLEMENTATION.....	23
4.1 Development Environment and Approach.....	23
4.2 Main System Flows.....	24
4.3 Key Implementations	24
4.3.1 Compatibility-Based Donor Matching (FR-06 — safety-critical)	24
4.3.2 Session Handling and Role-Based Access Control (FR-02)	25
4.3.3 AJAX/Fetch API Facade with Mock/Real Transport (FR-14, CPJ119 §2.6)	25
4.3.4 Image Handling — Database-Stored Photographs (FR-13, CPJ119 §2.5).....	25
4.3.5 Pagination and Reusable Presentation Components (FR-04).....	26
4.3.6 Shared Client-Side Validation (FR-14, CPJ119 §2.1/§4)	26
4.4 Administrative and Management Panels	26
4.5 API Surface	26
CHAPTER 5: TESTING AND RESULTS	28
5.1 Testing Strategy.....	28
5.2 Key Test Cases	28
5.3 Testing Evidence and Tools.....	30
5.4 Requirements Traceability	30
5.5 Evaluation Against the Assignment Brief.....	31
CHAPTER 6: PROJECT TIMELINE	32
CHAPTER 7: CONCLUSION, LIMITATIONS AND FUTURE SCOPE	34
7.1 Conclusion.....	34
7.2 Limitations	34
7.3 Future Scope	34
REFERENCES.....	36
APPENDICES	37
Appendix A: Blood Group Compatibility Chart.....	37
Appendix B: REST API Endpoint Reference	37
Appendix C: Database Schema.....	38
Appendix D: Selected Source Code.....	38
Appendix E: QA Evidence and Reproduction Steps.....	38
Appendix F: Additional Screenshots	39

LIST OF TABLES

Table 2-1: Comparison of Existing Blood Donor Management Systems

Table 2-2: Documented Limitations of Existing Practice and BloodBuddy's Response

Table 3-1: Entity Relationships and Cardinalities

Table 3-2: Core Data Dictionary

Table 3-3: Business Rules Enforced by the System

Table 4-1: Frontend Build Phases and Outcomes

Table 4-2: Summary of REST API Endpoints by Module

Table 5-1: Key Test Cases and Results

Table 5-2: Requirements-to-Implementation Traceability Matrix

Table 6-1: Phase-Wise Project Timeline and Completion Status

LIST OF FIGURES

- Figure 1-1: BloodBuddy Request Lifecycle
- Figure 3-1: N-Tier Architecture of BloodBuddy
- Figure 3-2: Use Case Diagram
- Figure 3-3: Entity Relationship Diagram
- Figure 3-4: Class Diagram of Core Entities
- Figure 3-5: Level-1 Data Flow Diagram
- Figure 4-1: Project File Structure (VS Code)
- Figure 4-2: Donor Search with Filters and Pagination
- Figure 4-3: Blood Request Form with Inline Validation
- Figure 4-4: Profile Photo Upload (Base64 Preview Round-Trip)
- Figure 4-5: Administrator Dashboard with Aggregate Analytics
- Figure 5-1: Automated QA Harness Report (266/266 Passing)
- Figure 6-1: Project Gantt Chart

LIST OF ABBREVIATIONS

AJAX	Asynchronous JavaScript and XML
API	Application Programming Interface
BICT	Bachelor of Information and Communication Technology
BLOB	Binary Large Object
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DAO	Data Access Object
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
ICT	Information and Communication Technology
JPA	Java Persistence API
JS	JavaScript
KPI	Key Performance Indicator
MVC	Model-View-Controller
NFR	Non-Functional Requirement
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
WHO	World Health Organisation

CHAPTER 1: INTRODUCTION

1.1 Overview

BloodBuddy is a web-based blood donor management system for the Nepali context, proposed in the CPP400 phase and developed for this module as a full-stack web application. The system serves four user roles. Donors register with their blood group, district, and contact details, manage their profile, and toggle their availability. Requesters search the donor registry by blood group and location, submit blood requests, and track each request through a status lifecycle. Hospital staff manage their institution's blood bank inventory and fulfil incoming requests from a dedicated portal. Administrators moderate requests, manage users, and monitor platform analytics through an administrative dashboard.

The platform supports Nepal's five partner hospitals — Tribhuvan University Teaching Hospital (TUTH), Bir Hospital, Patan Hospital, Manmohan Memorial Community Hospital, and Nepal Medical College Teaching Hospital (NMCTH) — as administrator-managed database records rather than hardcoded content, and the design provisions each with a dedicated, hospital-scoped API layer following the pattern `/api/hospitals/{hospitalId}/...` so that inventory and request data remain isolated per institution. The system's defining clinical rule — a donor may only be matched to recipients whose compatible donor groups include the donor's group — is derived from the blood group compatibility chart in Appendix A and is applied strictly in the recipient-to-donor direction; the frontend implements it as a single shared module so that every matching surface (request forms, donor dashboard, search) agrees, and the authoritative enforcement point is the backend service layer.

The front end is built with HTML, CSS, and vanilla JavaScript — no frameworks and no build step — and consumes the backend exclusively through AJAX/Fetch calls to REST endpoints via a single API facade, with shared client-side validation. The backend is a Spring MVC application organised into strict layers, with Hibernate/JPA persisting to MySQL and SMTP email for registration confirmations and urgent-request alerts.

1.2 Problem Statement

Nepal currently lacks a dedicated, reliable digital platform for blood donor discovery and management. Blood is a resource that cannot be manufactured and must come from voluntary donors, and the World Health Organisation notes that developing nations face the most severe shortages, largely due to the absence of organised donation management (WHO, 2023). In Nepal the existing approach is fragmented and informal, characterised by five persistent gaps:

- Blood requests are posted on Facebook and other social media platforms in an unstructured manner, with no guarantee of reaching compatible or available donors.
- There is no searchable, verified database of blood donors categorised by blood group and geographic location.
- Hospitals maintain manual blood bank records with no digital integration or real-time inventory visibility.
- There is no automated notification system to alert nearby donors when urgent requests are made.
- The absence of a centralised platform means that duplicate, fraudulent, or outdated requests cannot be moderated.

These gaps result in delayed responses during emergencies, wasted donor effort, and preventable loss of life. BloodBuddy exists to close these gaps with a structured, database-backed system whose critical logic — donor matching, request lifecycle, inventory integrity, and authorisation — lives in the backend.

1.3 Objectives

The general objective of this project was to design, implement, and test a secure web-based blood donor management system for Nepal using Spring MVC, Hibernate/JPA, and MySQL, satisfying the mandatory requirements of the CPJ119 assignment brief. The specific objectives were:

- To develop a searchable donor registry that allows users to find compatible blood donors by blood group and location in real time.
- To implement an automated email notification system that alerts nearby registered donors when an urgent blood request is submitted.
- To design a hospital portal for managing blood bank inventory with real-time updates.
- To develop and integrate dedicated local REST APIs for a minimum of five partner hospitals (TUTH, Bir Hospital, Patan Hospital, Manmohan Memorial Community Hospital, and NMCTH), enabling each institution to expose and consume blood inventory data through standardised, hospital-specific endpoints.
- To implement role-based access control distinguishing between Donors, Requesters, Hospital Staff, and Administrators.
- To expose all major system operations through a RESTful API architecture, tested and documented using Postman.
- To deliver a fully responsive frontend with client-side form validation, dynamic UI updates, and AJAX/Fetch communication with backend REST endpoints.
- To provide pagination for large result sets, including donor search results and administrative user and request listings.

1.4 Scope

In scope:

- A web-based application accessible via desktop and mobile browsers, targeting the Nepali population.
- Donor registration, profile management, and availability toggling.
- Blood request submission, tracking, and management by requesters, including a public emergency request flow for guests without an account.
- Donor search and filtering by blood group and location (district/city level).
- Hospital blood bank inventory management and request queue fulfilment.
- Local REST API integration with five partner hospitals (TUTH, Bir Hospital, Patan Hospital, Manmohan Memorial Community Hospital, and NMCTH), each with dedicated endpoints for inventory retrieval, stock updates, and blood request forwarding.
- Administrative controls for user management, request moderation, and system analytics.
- Automated email notifications triggered by registrations and blood requests.
- Client-side form validation and dynamic UI updates using JavaScript, with AJAX/Fetch API calls to communicate with backend REST endpoints.
- Pagination for large result sets, including donor search results and administrative user and request listings.

Out of scope:

- Real-time chat between donors and requesters; native mobile applications; and payment gateways — explicitly deferred to future work in the proposal.
- Live integration with hospital information systems; hospital and inventory data is admin-managed and seeded, and the local hospital APIs are designed but not yet connected to live institutional feeds.
- Clinical decision support of any kind; the compatibility chart is a matching filter, not medical advice.

1.5 Justification and Significance of the Study

Humanitarian contribution. An organised blood management system directly reduces the time between an urgent request and a successful donor match — the interval in which preventable deaths occur. Nepal's informal, social-media-based approach to blood requests is a documented public health concern, and a verified, searchable registry with automated alerts and hospital inventory visibility has clear, immediate social value.

Contribution to knowledge. The project demonstrates a hospital-scoped API isolation pattern (`/api/hospitals/{hospitalId}/...`) that lets multiple institutions share one platform without sharing each other's data — a pattern directly reusable in any multi-tenant health records system. It also demonstrates how a clinical rule set (blood group compatibility) can be centralised in one authoritative module so that several UI surfaces cannot drift into contradictory, unsafe matches, and how that rule must ultimately live in the server layer where requesters cannot influence it.

Contribution to methodology. The project traces every major feature from a documented pain point in Nepal's blood request workflow through formal requirement analysis, layered design, phased implementation, and evidence-based automated testing — a complete software development lifecycle within one academic module. It also aligns with Nepal's broader digital transformation goals: research by the Nepal Economic Forum indicates that 48% of Nepali firms lack any digital presence, and the country's healthcare and emergency response systems remain largely undigitised.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

This chapter reviews the concepts underlying BloodBuddy and evaluates existing systems in the same problem space, establishing the design decisions taken in Chapter 3. The review draws from academic sources, World Health Organisation publications, Nepal-focused digital economy reports, and publicly available analyses of existing donor platforms.

2.2 Related Concepts

2.2.1 Blood Donation Management

Blood donation management is the systematic process of recruiting, registering, screening, and coordinating blood donors to ensure an adequate supply for medical needs. The World Health Organisation defines an effective system as one that maintains a voluntary, non-remunerated donor base, ensures safe collection and storage, and enables rapid matching of donors to recipients (WHO, 2023). Traditional blood management in developing countries relies heavily on replacement donation — patients' families must provide donor substitutes — which is less reliable and more prone to safety risks than voluntary community donation supported by a searchable registry.

2.2.2 Web-Based Information Systems and Spring MVC

A web-based information system is hosted on a server and accessed through a browser, which suits healthcare and emergency contexts because it requires no installation, works across devices, and updates centrally. Spring MVC implements the Model-View-Controller pattern through a central DispatcherServlet that routes HTTP requests to annotated controllers (Spring, 2024), and its layered structure is widely adopted in enterprise-grade Java web development due to its modularity and scalability.

2.2.3 Hibernate ORM and Entity Relationships

Hibernate maps Java entity classes to relational tables without hand-written SQL, with JPA annotations expressing associations (@OneToOne, @OneToMany, @ManyToMany) (Bauer & King, 2015). For a donor management system, correctly modelled relationships are not cosmetic: foreign keys, cascade behaviour, and unique constraints are the foundation on which request ownership, inventory integrity, and per-hospital data isolation are enforced.

2.2.4 RESTful APIs and AJAX/Fetch Front Ends

RESTful design — stateless interactions over HTTP with resources addressed by URL and manipulated by methods (Fielding, 2000) — allows a decoupled JavaScript front end to consume the backend exclusively through Fetch calls, with JSON request/response bodies and consistent error shapes. Concentrating every call in one client-side API module keeps transport, error handling, and endpoint structure testable and swappable.

2.2.5 Web Security: Role-Based Access Control

Role-Based Access Control assigns permissions to roles rather than individuals, which is both scalable and aligned with enterprise security practice (Sandhu & Samarati, 1994). In BloodBuddy, RBAC ensures donors can only manage their own profiles, requesters only their own requests, hospital staff only their institution's inventory, and administrators the whole platform. Hiding UI elements is never mistaken for security: the authorisation decisions are

enforced server-side against the authenticated role, and secrets belong in environment variables, never in source code.

2.2.6 Image Storage Strategies

Three options exist: static files on disk (inflexible, lost on redeploy), external object storage/CDN (scalable, but stores nothing in the application database), and database BLOB/Base64 columns (data travels with the database; suited to small, fixed, low-volume images). The assignment brief mandates database-stored images (CPJ119 §2.5); BloodBuddy therefore stores profile photographs as Base64/BLOB with upload and retrieval managed through backend API endpoints.

2.3 Review of Existing Systems

2.3.1 Hamro Donor (Nepal)

Methodology/model used. Hamro Donor is one of the few existing attempts at a digital blood donor platform in Nepal. It operates as a mobile-first directory where donors register their blood group and contact information, backed by a basic cloud-hosted backend with no documented API layer.

Issues and limitations (for this project's context). The platform lacks a request management system — it provides donor contact details and leaves all coordination to the requester; there is no automated notification when a matching request is submitted; there is no administrative moderation, so outdated and unverified donor profiles accumulate; and hospital blood bank inventory is not integrated at all.

2.3.2 Sankalp India (Comparative Reference)

Methodology/model used. Sankalp is a well-established voluntary blood donation platform operating across India, with a structured backend, donor database, and request tracking. Automated SMS and email notifications and geolocation- and compatibility-based donor matching make it a useful benchmark for what a mature system can achieve.

Issues and limitations. The system is designed for the Indian context and does not address Nepali administrative divisions (districts, municipalities); it is not open source and cannot be adapted for local deployment in Nepal; and it does not provide a hospital-facing inventory management module.

2.3.3 The Manual Social-Media Workflow (Current Practice in Nepal)

Issues and limitations. Requests circulate as Facebook and WhatsApp posts with no structured matching or notification mechanism; hospitals keep blood bank records manually with no real-time visibility; requesters cannot confirm a donor's availability, compatibility, or location before making contact; and nobody can moderate duplicate, fraudulent, or outdated appeals (see Table 2-2).

Table 2-1: Comparison of Existing Blood Donor Management Systems

Feature	Hamro Donor	Sankalp India	Manual Social-Media Workflow	BloodBuddy (Implemented)
Donor registry	Directory only, unverified	Verified, structured	None (ad-hoc posts)	Verified registry, availability toggle, profile management
Request management	None — requester coordinates	Yes	None (Facebook/WhatsApp posts)	Full lifecycle: Pending → Matched → Fulfilled, tracked per requester

Compatibility matching	None	Group + proximity	None	Appendix A chart applied recipient→donor on every matching surface
Automated notifications	None	SMS + email	None	SMTP email on registration and urgent requests
Hospital inventory module	None	None	Manual paper registers	Per-hospital blood bank portal with stock levels and fulfilment
Moderation / administration	None	Internal	None	Admin moderation, user management, request forwarding, analytics
Hospital-scoped API	None	None	None	/api/hospitals/{hospitalId}/... with per-institution data isolation
Suited to Nepali context	Partially	No (India-specific)	Status quo	Yes — districts, Nepali partner hospitals, mobile-first responsive web

Table 2-2: Documented Limitations of Existing Practice and BloodBuddy's Response

Documented Limitation	BloodBuddy Response
Requests posted informally with no guarantee of reaching compatible donors	Searchable registry filtered by blood group, district, and live availability; compatibility chart applied to every match
No structured request lifecycle	Request entity with status transitions (Pending → Matched → Fulfilled; Cancelled/Rejected terminal), timeline entries, and per-requester tracking
No automated donor alerting	EMAIL_NOTIFICATION entity + SMTP delivery on registration and urgent requests (implementation scheduled; design Chapter 3)
Manual hospital blood bank records	BLOOD_INVENTORY per hospital with staff-managed stock levels, fulfilment-driven decrement, and request queues
No moderation of duplicate/fraudulent appeals	Administrator moderation with request forwarding, user suspension, and platform analytics
Donor contact data goes stale	Availability toggling, admin suspension of inactive accounts, and moderation of outdated profiles

2.4 The Market Gap and BloodBuddy's Position

The evidence establishes a gap narrower than “Nepal lacks blood donor software”: the missing piece is an affordable, locally relevant platform that reliably registers donors, matches requests by compatibility and location, notifies donors automatically, gives hospitals real-time inventory control, and moderates abuse — without requiring infrastructure that a Nepali institution cannot sustain. Hamro Donor shows the cost of a directory without a request lifecycle; Sankalp shows the value of one but does not transfer to Nepal. BloodBuddy combines the verified registry, automated notifications, request management, and hospital inventory integration in a single system purpose-built for Nepal's districts and institutions, consistent with evidence that the country's digital gap is most acute in public service domains (NIPoRe, 2023).

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Requirements Summary

The functional requirements finalised in the proposal are carried forward as the contract this implementation must satisfy, numbered FR-01 to FR-14 for traceability; Section 5.4 traces each to its implementation and test evidence. The core functions are: user registration and authentication (FR-01); role-based access control across four roles (FR-02); the donor registry with availability toggling (FR-03); donor search and filtering with pagination (FR-04); blood request submission and tracking (FR-05); compatibility-based donor matching (FR-06); a public emergency flow with guest status lookup (FR-07); hospital inventory management (FR-08); hospital request queues and fulfilment (FR-09); hospital-scoped local APIs for five partner hospitals (FR-10); administration and moderation (FR-11); SMTP notifications (FR-12); database-stored images (FR-13); and AJAX/Fetch communication with a responsive, validated frontend (FR-14). Non-functional requirements NFR-01 to NFR-06 cover usability, security, reliability, maintainability, performance, and auditability. Two requirements deserve design emphasis: FR-06 (compatibility-based matching, the system's most safety-critical rule) and FR-10 (per-hospital data isolation).

3.2 System Architecture

BloodBuddy's backend is organised into the mandatory N-Tier layers. No layer skips its neighbour: controllers never touch repositories, and repositories contain no business logic. Donor matching, request status transitions, and inventory decrements are service-layer responsibilities; the frontend handles presentation and basic input validation only.

Figure 3-1: Proposed System Architecture

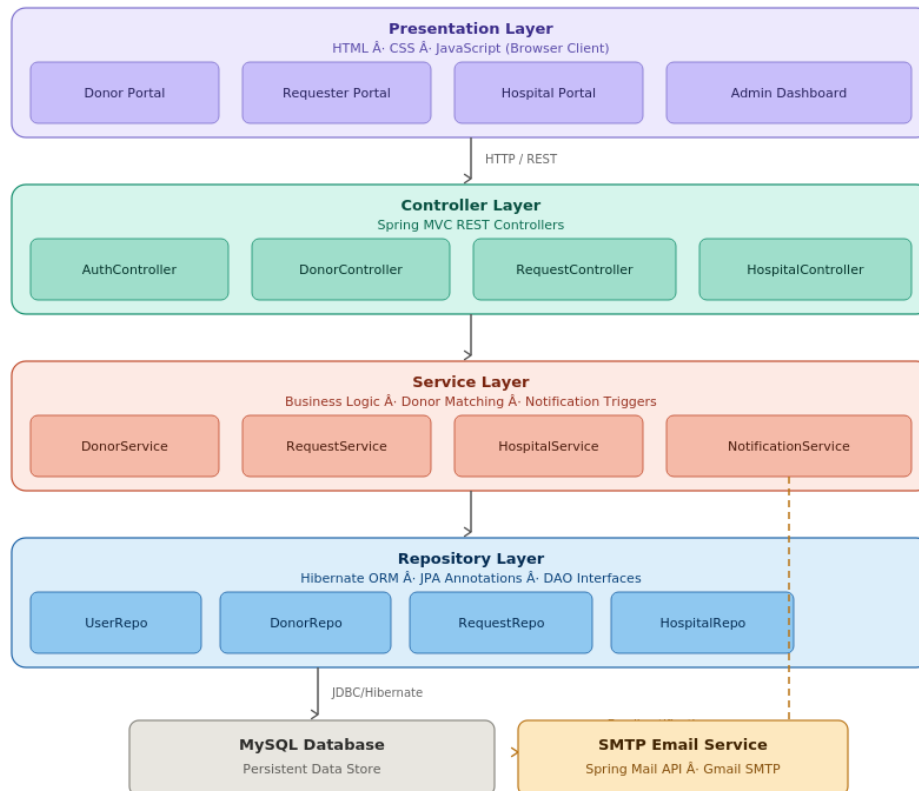


Figure 3-1: N-Tier Architecture of BloodBuddy

Package structure:

```
com.bloodbuddy    application root
controller/       AuthController, DonorController, RequestController,
                  HospitalController, InventoryController, AdminController, ProfileController – one
                  hospital-scoped controller module per partner hospital
service/          matching *Service / *ServiceImpl per domain: DonorSearchService,
                  BloodRequestService, InventoryService, NotificationService, AdminService
repository/       matching *Repository interfaces (Spring Data JPA)
model/            User, Donor, BloodRequest, Hospital, BloodInventory, EmailNotification
dto/              request/response bodies (entities never exposed directly)
security/         Spring Security configuration, role rules, password encoding
exception/        GlobalExceptionHandler (@ControllerAdvice),
                  ResourceNotFoundException, ValidationException
```

The presentation layer as built is a framework-free HTML/CSS/JavaScript application of 23 pages organised into public pages (landing, about, contact, legal, 404), authentication pages, and four role portals, with five shared JavaScript modules: a session-aware navigation and role-guard module (nav.js), a validation module (bb-validate.js), the API facade (bb-api.js), a reusable pagination component (bb-paginate.js), a shared demo store (bb-store.js), and the compatibility module (bb-compat.js).

[INSERT SCREENSHOT: Figure 4-1 — VS Code explorer showing CODE/HTML, CODE/js (nav.js, bb-api.js, bb-validate.js, bb-paginate.js, bb-compat.js, bb-store.js), CSS, tools, resptest]

Figure 4-1: Project File Structure (VS Code)

3.3 Use Case Model

[INSERT DIAGRAM: Figure 3-2 — use case diagram: Donor (register, manage profile and availability, view compatible open requests, accept/decline, view history); Requester (register, login, search donors, submit request, submit public emergency request, track requests, cancel); Hospital Staff (login, manage inventory, review queue, accept/fulfil requests, update stock); Administrator (manage users, moderate/forward requests, view analytics, reset demo data); Guest (search, emergency request, status lookup by request ID)]

Figure 3-2: Use Case Diagram

3.4 Entity Relationship Design

Figure 3-3: Entity Relationship Diagram

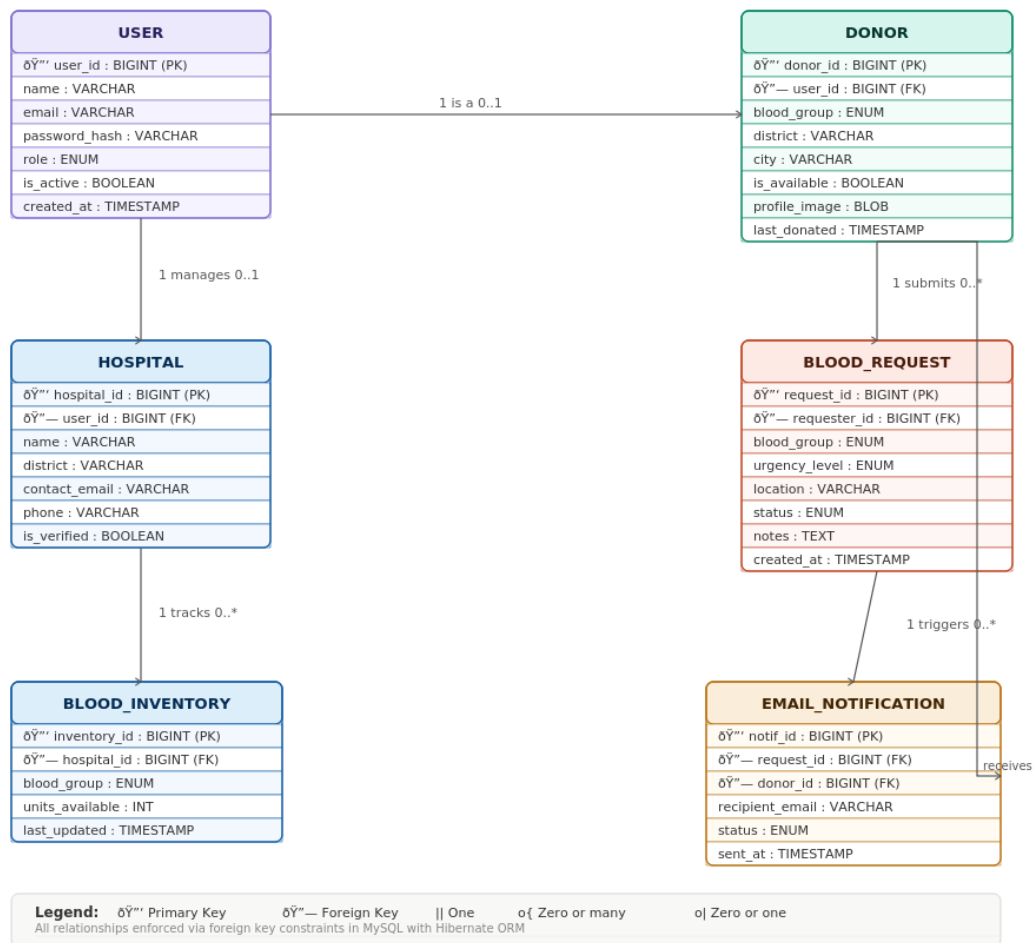


Figure 3-3: Entity Relationship Diagram

Table 3-1: Entity Relationships and Cardinalities

Relationship	Cardinality	Notes
User → Donor	1:1	A Donor profile is a role-scoped extension of a User account
User → BloodRequest	1:N	A requester owns many requests
Donor → BloodRequest	1:N	A donor may serve multiple requests; each request is served

		by at most one donor (matched at accept)
Hospital → BloodInventory	1:N	Per-hospital stock rows, one per blood group
Hospital → BloodRequest	1:N	Requests routed/forwarded into a hospital's queue
BloodRequest → EmailNotification	1:N	Each notification references the request that triggered it
Donor ↔ Hospital (affiliation)	M:N	A donor is registered with the partner hospitals in their own district — "the blood banks near you". Realised as the donor_hospital_affiliation join table (CPJ119 §2.3)
Donor ↔ Blood Group (compatibility)	M:N	Realised through the Appendix A compatibility chart; drives donor matching. A compatibility rule, not a physical join table
User → Role	N:1	Role enum: DONOR / REQUESTER / HOSPITAL / ADMIN

Table 3-2: Core Data Dictionary

Field	Type	Description
userId	Long (PK)	Unique user identifier; name, email, phone, role enum (DONOR/REQUESTER/HOSPITAL/ADMIN), status (ACTIVE/SUSPENDED)
donorId	Long (PK)	Donor profile: blood group enum, district, availability flag, last-donation data; Base64/BLOB photograph
requestId	Long (PK)	Unique request; blood group, units, needed-by date, urgency, district, hospital, contact; status (PENDING/MATCHED/FULFILLED/CANCELLED/REJECTED); timeline entries; guest requests carry an EM- public identifier
hospitalId	Long (PK)	Partner hospital; name, type, district, blood-bank-open flag
inventoryId	Long (PK)	Stock row: hospital_id FK + blood group, units available, updated timestamp
notificationId	Long (PK)	Notification record: recipient, type (REGISTRATION/URGENT_REQUEST), request FK, delivery status

3.5 Class Design

The class structure inherits from an abstract User base class specialised by role, with the request, inventory, and notification classes associated to it, and service interfaces encapsulating the business operations (DonorSearchService, BloodRequestService, InventoryService, NotificationService).

Figure 3-2: Class Diagram

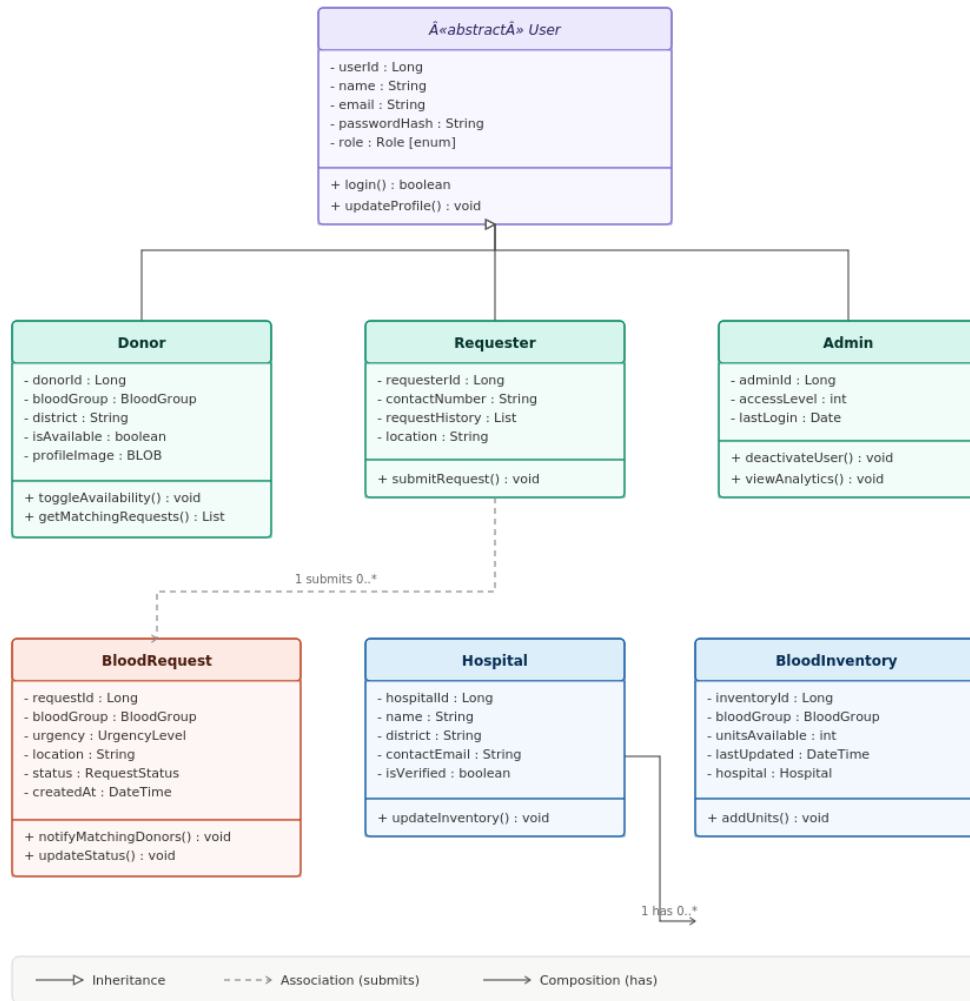


Figure 3-4: Class Diagram of Core Entities

3.6 Data Flow

[INSERT DIAGRAM: Figure 3-5 — Level-1 DFD: processes 1 User & Role Management, 2 Donor Registry & Search, 3 Blood Request Management, 4 Inventory Management, 5 Notification Management, 6 Administration & Moderation; external entities: Donor, Requester, Hospital Staff, Administrator, SMTP server]

Figure 3-5: Level-1 Data Flow Diagram

3.7 Business Rules

Table 3-3: Business Rules Enforced by the System

#	Rule	Enforcement
BR-1	A donor may only be matched to requests whose recipient group lists the donor's group as compatible	Appendix A chart applied in the recipient→donor direction in one shared module; authoritative check in the service layer
BR-2	A request follows Pending → Matched → Fulfilled; Cancelled	Status transition guard in the

	and Rejected are terminal and cannot change	request service
BR-3	A request can be matched by at most one donor	Single donor stamp on accept; subsequent accepts rejected (unique match)
BR-4	Fulfilment decrements the hospital's inventory for that blood group and never below zero	InventoryService transactional decrement with floor check
BR-5	A hospital's staff may write only that hospital's inventory and queue	Hospital-scoped endpoints keyed by hospitalId; route rules in Spring Security
BR-6	Only Administrators moderate or forward requests and manage users	Admin-only endpoints and moderation actions behind role checks
BR-7	Donors see only open requests they can serve; a decline hides the request for that donor only and never mutates the request	Per-donor filtered query + per-donor decline list
BR-8	Guest emergency lookups expose guest request IDs only	Public lookup rejects member (BB-) request identifiers
BR-9	No plaintext or exposed credential data	Password hashing; DTOs exclude credential fields; secrets via environment variables

3.8 Design Refinements from Proposal to Implementation

Four design details were refined between the proposal-stage design and the implemented system, in the normal course of testing and hardening — recorded here for transparency:

- Single API facade. The proposal specified AJAX/Fetch calls to REST endpoints; implementation concentrated every call in one shared module (bb-api.js) that mirrors the proposal's Appendix B endpoints, so pages never touch the transport directly and the swap from the demonstration store to the real Spring backend is a single configuration flag.
- Compatibility rule centralised. The Appendix A chart was initially duplicated in the two request forms; donor matching would have required a third copy, so it was extracted into one shared module (bb-compat.js) with the matching direction documented — recipient → compatible donor groups — after testing showed that reading the chart backwards would offer a donor a patient they must not give blood to.
- Hospital name normalisation. The five partner hospitals are referred to by several natural variants (abbreviations and full names) across the interface; an alias table normalises every submitted request to canonical hospital labels so that requests forwarded or submitted under variant names land in the correct hospital queue's equality filter.
- Status vocabulary finalised. Request statuses were finalised as Pending → Matched → Fulfilled, with Cancelled and Rejected as terminal outcomes, each status change recorded as a timeline entry on the request — replacing the proposal's informal “open/closed” wording and aligning the tracker, hospital queue, and moderation views on one vocabulary.

3.9 Security Design

- Role-based access control across four roles. In the demonstration build, role-private pages are guarded client-side: logged-out visitors are bounced to the login page and wrong-role users to their own dashboard, with the blocked URL stashed so login can continue to the original target — but only if the role selected at login may actually access it.
- Server-side enforcement by design. The authoritative gate is Spring Security in the service layer: route rules make `/api/auth/**` public, `/api/admin/**` administrator-only, and `/api/hospitals/{hospitalId}/**` accessible only to that hospital's staff, so hiding frontend elements is never mistaken for security.
- Hospital data isolation. Every hospital-scoped endpoint is keyed by `hospitalId` and scoped to that institution's inventory and queue only, per requirement FR-10.
- Validation on both ends. Shared client-side validation covers email, phone, blood group format, password strength, future dates, units, and required consent; corresponding server-side validation is part of the scheduled backend, per the assignment's requirement that input validation is required on both the frontend and backend.
- Guest privacy guard. The public emergency status lookup answers guest request IDs (EM-...) but explicitly rejects member request IDs (BB-...), so the public endpoint cannot be used to read members' requests.
- Secrets management. SMTP and database credentials are supplied via environment variables — never hardcoded (CPJ119 §2.4).

CHAPTER 4: IMPLEMENTATION

4.1 Development Environment and Approach

Frontend: HTML, CSS, and vanilla JavaScript in VS Code — no frameworks and no build step — with quality verification driven by headless Chrome automation. Backend (scheduled implementation phase): Java + Spring Boot (Spring MVC, Spring Security, Spring Data JPA, Spring Mail, Validation, Lombok), MySQL, IntelliJ IDEA. API testing: Postman. A Python test stub serves the static site and an Appendix B REST stub on one port so the AJAX/Fetch layer can be exercised over real HTTP before the Spring backend exists.

Implementation followed a dependency-ordered roadmap (Table 4-1), with the automated QA harness run at the end of every session before proceeding — most consequentially after the request lifecycle and after the real-mode transport work.

Table 4-1: Frontend Build Phases and Outcomes

Phase	Deliverable	Key Outcome
0	Project setup, shared UI conventions, page scaffolding	22→23 pages; consistent hero/nav/footer system; brand styles
1	Shared modules: bb-validate.js, bb-api.js, bb-paginate.js, bb-compat.js, demo store	One validation rule set; one API facade; one compatibility chart; one pager
2	Auth pages + session-aware navigation + RBAC page guards	Role-aware nav on every page; private pages bounce correctly; return-URL continuation
3	Donor search: filters, sort, widen fallback, pagination	Group/district/availability actually filter; emergency search never dead-ends at zero results
4	Request forms (member + public emergency) with inline validation	Both forms validated; compatibility chips; hospital normalised through alias table
5	Request lifecycle: accept/decline, Matched status, fulfilment	Donor matching in correct chart direction; fulfilment decrements inventory; timeline entries
6	Hospital portal: inventory management + request queue	Stock adjustment; queue statuses labelled; open pill mirrors profile toggle
7	Administrator portal: analytics, user management, moderation	Live KPI counts; suspend; forward flow; Emergency badges; demo reset
8	Four profile editors + Base64 photo upload + dirty-state guard	Consistent save bar/discard/toast; photo round-trip verified
9	Public site: landing, about, contact, terms, privacy, 404	Legal pages site-wide; branded 404 made responsive
10	QA harness: automated functional checks	266/266 checks passing; five real defects caught and fixed
11	Responsive verification at 360/430/768/1280	92/92 page×width combinations without horizontal overflow
12	Real-mode transport verification over HTTP	16/16 checks against REST stub; /api/api regression caught, fixed,

		negative-tested
--	--	-----------------

4.2 Main System Flows

The request lifecycle (Figure 1-1) is the system's backbone: a requester searches donors, submits a request (or a guest uses the public emergency form), the request enters the administrator's moderation queue and is forwarded to a partner hospital, compatible donors see and accept it (status Matched), and the hospital marks it fulfilled — closing the request and decrementing that hospital's stock for the blood group. Every state change is appended to the request's timeline and visible to the requester in the tracking view.

[INSERT DIAGRAM: Figure 1-1 — Requester submits → Pending → Admin forwards → Hospital queue → Donor accepts (Matched) → Hospital fulfils (Fulfilled, stock decremented); Cancelled/Rejected terminal; timeline entries at every step]

Figure 1-1: BloodBuddy Request Lifecycle

The donor discovery flow (landing → search → donor cards → request modal → tracking) preserves the built page sequence: search calls the donor search endpoint with blood group, district, and availability filters; results are paginated; the request modal creates the request through the API facade and redirects to tracking. The public emergency flow mirrors it for guests, issuing an EM- identifier and a status-lookup card.

[INSERT SCREENSHOT: requester-search.html — filter chips, donor cards, pager]

Figure 4-2: Donor Search with Filters and Pagination

[INSERT SCREENSHOT: requester-request.html — compatibility chips, future-date field, inline errors]
[INSERT SCREENSHOT: emergency-request.html — public guest form with 102 reminder and success ID]

Figure 4-3: Blood Request Form with Inline Validation

4.3 Key Implementations

4.3.1 Compatibility-Based Donor Matching (FR-06 — safety-critical)

The Appendix A compatibility chart is implemented once, in `bb-compat.js`, and read in the correct clinical direction — recipient → compatible donor groups. To decide whether a donor may serve a request, the code asks `isCompatible(donorGroup, recipientGroup)`, never whether the recipient's group appears in `donorsFor(donorGroup)`; reading the chart backwards would offer a donor a patient they must not give blood to. The donor dashboard therefore lists only open requests the logged-in donor can actually serve:

```
// CODE/js/bb-compat.js (excerpt) — recipient → compatible DONOR groups (proposal
Appendix A)
var COMPAT = { 'A+': ['A+', 'A-', 'O+', 'O-'], 'A-': ['A-', 'O-'], 'B+': ['B+', 'B-', 'O+', 'O-'],
               'B-': ['B-', 'O-'], 'AB+': ['AB+', 'AB-', 'A+', 'A-', 'B+', 'B-', 'O+', 'O-'],
               'AB-': ['AB-', 'A-', 'B-', 'O-'], 'O+': ['O+', 'O-'], 'O-': ['O-'] };
// donor may serve request only if donor's group is in COMPAT[recipientGroup]
BloodBuddyCompat.isCompatible(donorGroup, recipientGroup);
BloodBuddyCompat.forRecipient(g); // request forms: live compatible-group chips
```

The same rule is verified by the automated suite (Section 5.2, TC-09): a B+ donor is offered B+ and AB+ recipients and never an A+ one. The module's header records that the authoritative compatibility rule belongs in the Spring service layer, so a requester can never talk the frontend into an unsafe match.

4.3.2 Session Handling and Role-Based Access Control (FR-02)

The demonstration build implements RBAC end-to-end at the page level: a session record ({role, name}) set at login/register drives a role-aware navigation rebuilt on every page; a guard map declares each private page's permitted role; logged-out visitors bounce to the login page and wrong-role users to their own dashboard; the blocked URL is stashed in session storage and the login page continues to it only if the selected role may access it; and in-page calls-to-action are swapped per role, so a donor is never offered a requester action such as “Request blood”:

```
// CODE/js/nav.js (behaviour summary)
RBAC_GUARD = { 'donor-dashboard.html': ['donor'], 'hospital-dashboard.html':
['hospital'],
  'admin-dashboard.html': ['admin'], 'requester-search.html':
['requester'], ... }
BloodBuddyNav.canAccess(role, page) // login continue-to honoured only if permitted
// wrong role → own dashboard; logged out → auth-login.html with stashed return URL
```

The corresponding server-side enforcement — Spring Security route rules and role checks on every write endpoint — is specified in Section 3.9 and scheduled with the backend.

4.3.3 AJAX/Fetch API Facade with Mock/Real Transport (FR-14, CPJ119 §2.6)

Every page reaches the backend through one facade (bb-api.js) that mirrors the proposal's Appendix B endpoints. In mock mode it serves calls from the shared demonstration store with simulated latency; flipping one flag (MOCK = false) routes the identical calls through real fetch() to the Spring endpoints, so pages keep working unchanged:

```
// CODE/js/bb-api.js (excerpt) — real transport; BASE is the origin prefix only
function realRequest(method, path, body) {
  return fetch(BASE + path, {
    method: method,
    headers: body ? { 'Content-Type': 'application/json' } : undefined,
    body: body ? JSON.stringify(body) : undefined,
    credentials: 'same-origin'
  }).then(function (res) { /* normalise errors */ });
}
// every path already carries /api/, so BASE must stay '' — a '/api' prefix
// would compose /api/api/donors/search and 404 every call
```

The commented rule above is not decorative: the first real-mode run composed /api/api/... and 404'd every endpoint with no JavaScript error — the failure was invisible in mock mode because mock mode never reaches the concatenation. It was found by running the fetch path against the REST stub (Section 5.2, TC-20), fixed, and then guarded by a dedicated automated check that was negative-tested (re-injecting the bad prefix demonstrably fails the suite).

4.3.4 Image Handling — Database-Stored Photographs (FR-13, CPJ119 §2.5)

All four role profile editors implement secure photo upload exactly as the assignment's database-image requirement anticipates: a file input restricted to JPG/PNG, read client-side via FileReader into a Base64 data URL, rendered as a live preview with a remove action, and

carried as the photo field of the profile payload saved through the API facade — never written to the project directory as a static file. The scheduled backend stores the image as a BLOB column (or the equivalent Base64 string) and serves retrieval through an authenticated API endpoint, satisfying the brief's “stored within the database” constraint end-to-end.

[INSERT SCREENSHOT: donor-profile.html — photo preview, Remove control, save bar]

Figure 4-4: Profile Photo Upload (Base64 Preview Round-Trip)

4.3.5 Pagination and Reusable Presentation Components (FR-04)

A single reusable pager component (bb-paginate.js) serves every listing surface — donor search results, the admin user table, the admin moderation queue, and the hospital queue — with previous/next controls, a page-size selector (5/10/20), and automatic hiding when a list is empty. Pages call `setTotal(filtered)` and `slice(items)` in their render path; when the backend arrives, the client-side slice is replaced by `?page=&size=` query parameters without touching the pages.

4.3.6 Shared Client-Side Validation (FR-14, CPJ119 §2.1/§4)

Validation previously existed as nine copies of ad-hoc alert-based checks; it is now one shared module (bb-validate.js) providing email, phone, blood group format, password strength, future-date, units, and consent rules, rendered as inline field-level errors under each control. A computed-style pass verified that every error renders visibly (6.03:1 contrast against its background, passing WCAG AA) and that the error ring styling wins the cascade on every form — evidence in Section 5.2 (TC-01) and Appendix E.

4.4 Administrative and Management Panels

The build comprises 23 pages: public pages (landing, about, contact, terms, privacy, branded 404), authentication pages (login, register, forgot password), and a portal per role. All four portals follow a consistent pattern — hero with role imagery, role-aware navigation, cards for primary data, and status badges (Pending/Matched/Fulfilled; Emergency for guest requests). The administrator portal provides KPI tiles with live counts, user management with suspension and a demo-data reset control, and request moderation with status tabs, a hospital filter (including “not routed yet”), and forwarding to partner hospitals. The hospital portal provides inventory management with stock adjustment and a request queue with accept/fulfil actions; the requester portal adds tracking tabs per status with a live open-request count.

[INSERT SCREENSHOT: admin-dashboard.html — KPI tiles, users table with pagination, reset control]

[INSERT SCREENSHOT: admin-requests.html — moderation queue, hospital filter, forward action]

[INSERT SCREENSHOT: hospital-dashboard.html — inventory table with stock adjustment]

[INSERT SCREENSHOT: requester-tracking.html — status tabs, timeline, live open-count]

Figure 4-5: Administrator Dashboard with Aggregate Analytics

4.5 API Surface

Table 4-2: Summary of REST API Endpoints by Module

Module	Representative Endpoints	Access
Auth	POST /api/auth/register, POST /api/auth/login, POST /api/auth/forgot	Public

Donors	GET /api/donors/search (group/district/availability, paginated), GET /api/donors/{id}	Auth
Requests	GET /api/requests (?requester=&donor=&hospital=&guest=), POST /api/requests, GET /api/requests/{id}, PATCH /api/requests/{id} (status / moderation / fulfilment)	Auth (role-scoped)
Hospital inventory	GET /api/hospitals/inventory, PUT /api/hospitals/inventory/{bloodGroup}	Hospital staff
Hospital-scoped (local APIs)	GET/PUT /api/hospitals/{hospitalId}/inventory[{bloodGroup}], POST /api/hospitals/{hospitalId}/requests, GET /api/hospitals/{hospitalId}/requests/{requestId}/status, GET /api/hospitals/{hospitalId}/donors/nearby	That hospital only
Admin	GET /api/admin/users, PATCH /api/admin/users/{id} (status), moderation/forward via PATCH /api/requests/{id}	Administrator
Profile	GET /api/profile/{role}, PUT /api/profile/{role} (→ GET /api/auth/me + role-scoped PUT in final backend)	Auth (own profile)

Every page consumes exactly these endpoints through the facade; the demonstration store implements them in mock mode, and the scheduled Spring controllers implement the same contract. (Full request/response listing in Appendix B; the exported Postman collection is submitted with the backend phase.)

CHAPTER 5: TESTING AND RESULTS

5.1 Testing Strategy

Testing was continuous and checkpoint-based: the automated QA harness was run at the end of every build session, and no session's work was accepted while a check failed. The verification approach has three layers, each covering what the others structurally cannot:

- Functional harness (resptest/qa-harness.html): a self-driving test page that loads every page in isolated frames with cleared storage, seeds controlled data, exercises the interactions of all four roles, and reports per-check results — 266 checks.
- Responsive layout check (resptest/responsive-check.html): loads all pages at 360, 430, 768, and 1280 px viewports and fails any page whose content overflows horizontally — 92 page×width combinations.
- Real-mode transport check (resptest/real-mode-check.html + tools/mock_api_server.py): with the facade's mock mode switched off and the data store pinned empty, five representative pages must render their rows purely from HTTP responses served by a REST stub — proving the AJAX/Fetch layer actually performs network calls rather than merely containing them — 16 checks.

The suite is deliberately adversarial towards itself: guards were negative-tested (deliberately re-injecting a known-bad configuration must fail the suite), and the harness seeds its own data rather than depending on other pages' leftovers. Backend API testing in Postman — success and failure cases per module, with the collection exported as JSON — is part of the scheduled backend phase; no backend test is reported as passed here.

5.2 Key Test Cases

Table 5-1: Key Test Cases and Results

ID	Test	Expected	Result
TC-01	Register/login with invalid input	Inline field errors, submit blocked, no session created	PASS
TC-02	Logged-out access to a role-private page	Redirect to login with return URL preserved	PASS
TC-03	Wrong-role access to a private page	Bounced to own dashboard; no cross-role access	PASS
TC-04	Donor search filters (group/district/availability)	Results filtered; filter state persists; sort works	PASS
TC-05	Search widen fallback for empty district	Group shown across districts with explanation; no dead-end	PASS
TC-06	Member request submit → tracking → cancel	POST /api/requests persists; appears in tracker; cancellable while open	PASS
TC-07	Guest emergency submit + status lookup	EM- ID issued and displayed; lookup returns status; member	PASS

		(BB-) IDs rejected	
TC-08	Compatibility direction (B+ donor)	Offered B+/AB+ requests only; never an A+ recipient	PASS
TC-09	Donor accept	Status Matched, donor stamped, timeline entry; hospital queue shows "Donor matched"	PASS
TC-10	Hospital fulfilment	Request Fulfilled; hospital inventory decremented for that group	PASS
TC-11	Donor decline	Hidden for that donor only; request unchanged for others	PASS
TC-12	Admin moderation and forwarding	Request routed to chosen hospital queue; Emergency badge on guest requests	PASS
TC-13	Inventory stock adjustment	Level updated; blood-bank-open pill mirrors profile toggle	PASS
TC-14	User suspension (admin)	Status change reflected in user table	PASS
TC-15	Pagination across listings	Page sizes 5/10/20; Prev/Next; pager hides when empty	PASS
TC-16	Profile photo upload (Base64)	Preview renders; remove works; JPG/PNG-only enforced	PASS
TC-17	Responsive layout, 23 pages × 4 widths	No horizontal overflow at 360/430/768/1280	PASS (92/92)
TC-18	Real-mode transport over HTTP	Pages render data purely from HTTP responses; endpoints 200 in stub log	PASS (16/16)
TC-19	Transport guard negative test	Re-injecting BASE='/api' fails the suite (266→264); restoring passes	PASS
TC-20	Backend API suite in Postman (success + failure, concurrent accepts)	One 200, one conflict per TC-03 analog; collection exported	Pending (backend phase)
TC-21	SMTP registration confirmation email	Email received; credentials from environment variables	Pending (backend phase)

[INSERT SCREENSHOT: resptest/qa-harness.html — per-check PASS report in headless Chrome]
 [INSERT SCREENSHOT: resptest/responsive-check.html — 92/92 page×width matrix]
 [INSERT SCREENSHOT: real-mode-check.html — 16/16 over HTTP with the stub's GET /api/... -> 200 log]

Figure 5-1: Automated QA Harness Report (266/266 Passing)

5.3 Testing Evidence and Tools

The harness runs headlessly in Chrome with a fresh profile and cache buster, dumps its results to the DOM, and exits non-zero on failure, making it repeatable from the command line (the exact commands are recorded in Appendix E). The visual validation pass used a computed-style probe rather than screenshots alone: every inline error on both request forms was measured for display, box size, colour contrast (6.03:1, WCAG AA), and ring styling, with the probe and its screenshots kept as re-runnable artefacts. Five genuine defects the suite caught — a dead search route, a status-crash on unlabelled states, a transport prefix that would have 404'd every real call, a botched page edit, and a non-responsive 404 page — are recorded with their fixes in Section 7.2's context and Appendix E.

5.4 Requirements Traceability

Table 5-2: Requirements-to-Implementation Traceability Matrix

Requirement	Implementation	Evidence
FR-01 Registration/authentication	Auth pages through API facade; server-side auth scheduled	TC-01, TC-02
FR-02 RBAC (4 roles)	Role-aware nav + page guards; Spring Security rules specified (§3.9)	TC-02, TC-03
FR-03 Donor registry + availability	Donor profiles, availability toggle, admin suspension	TC-04, TC-14
FR-04 Search + filtering + pagination	Donor search endpoint via facade; shared pager	TC-04, TC-05, TC-15
FR-05 Request submission + tracking	Request forms, tracker tabs, timeline entries	TC-06
FR-06 Compatibility matching	bb-compat.js (recipient→donor direction); service-layer rule specified	TC-08, TC-09
FR-07 Public emergency flow	Guest form, EM- IDs, public status lookup with privacy guard	TC-07
FR-08 Hospital inventory management	Inventory table, stock adjustment, open/closed pill	TC-13
FR-09 Hospital queue + fulfilment	Accept/fulfil actions; stock decrement on fulfil	TC-09, TC-10
FR-10 Hospital-scoped local APIs	/api/hospitals/{hospitalId}/... contract via facade + stub	TC-18; Appendix B
FR-11 Administration + moderation	Admin KPIs, user management, moderation + forwarding	TC-12, TC-13, TC-14
FR-12 SMTP notifications	EMAIL_NOTIFICATION design (§3.4); Spring Mail scheduled	Pending (backend phase)
FR-13 Database-stored images	Base64 upload + preview via facade; BLOB storage scheduled	TC-16
FR-14 AJAX/Fetch + validation +	API facade; shared validation;	TC-01, TC-17, TC-18

responsive	responsive CSS	
NFR-01–06 Usability, security, reliability, maintainability, performance, auditability	Responsive UI; RBAC + dual-end validation; consistent error shapes; layered design + shared modules; pagination; timeline audit trail	TC-01–TC-19; Sections 3.7–3.9

5.5 Evaluation Against the Assignment Brief

Every mandatory §2.1 frontend criterion is implemented and evidenced: semantic HTML/CSS/JS with a fully responsive layout (92/92 layout checks), client-side form validation (shared module, inline errors, TC-01), dynamic UI updates, basic AJAX/Fetch integration with backend APIs (proven over real HTTP, TC-19), search and filtering, pagination, role-based access control, and dashboard analytics. Code quality criteria are addressed through the strict layered design, shared modules, consistent naming, global error shapes, and validation on both ends (client-side verified; server-side scheduled).

The remaining mandatory criteria belong to the scheduled backend phase and are not claimed as complete: the Spring MVC N-Tier implementation (§2.2), MySQL + Hibernate persistence with 1:1, 1:N and N:N relationships (§2.3), SMTP registration confirmation via environment-variable credentials (§2.4), the backend half of database-stored images (§2.5 — the frontend Base64 upload is done), and the Postman collection exported as JSON (§2.6 — the endpoint contract is implemented and HTTP-verified against the stub, but the collection itself is exported with the backend). The demonstration build's persistence is browser-based and is explicitly not presented as the backend.

CHAPTER 6: PROJECT TIMELINE

The project followed the phase-dependency order of the build roadmap (Table 4-1), with the automated QA checkpoint passing at the end of every phase before the next began. Periods are shown as project weeks; the backend phase is scheduled, not yet performed.

Table 6-1: Phase-Wise Project Timeline and Completion Status

Phase	Activity	Period	Status
0–1	Proposal (CPP400) approval; requirements finalisation; project setup and shared UI conventions	[Week 1]	Complete
2	Shared modules: validation, API facade, pagination, compatibility chart, demo store	[Weeks 1–2]	Complete
3	Auth pages, session-aware navigation, RBAC page guards	[Week 2]	Complete
4–5	Donor search + request forms (member and public emergency) with inline validation	[Week 3]	Complete
6–7	Request lifecycle (accept/match/fulfil) + hospital inventory and queue	[Weeks 3–4]	Complete
8	Administrator portal: analytics, user management, moderation	[Week 4]	Complete
9	Profile editors, Base64 photo upload, legal and public pages	[Week 5]	Complete
10	Validation/fetch/pagination consolidation; visual and contrast pass	[Weeks 5–6]	Complete
11	QA harness expansion; defect fixing	[Week 6]	Complete
12	Responsive verification; real-mode transport verification	[Weeks 6–7]	Complete
13	Spring MVC backend: entities, relationships, security, SMTP	[Weeks 8–10]	Scheduled (backend phase)
14	Postman collection, final integration testing, report finalisation	[Weeks 11–12]	In progress

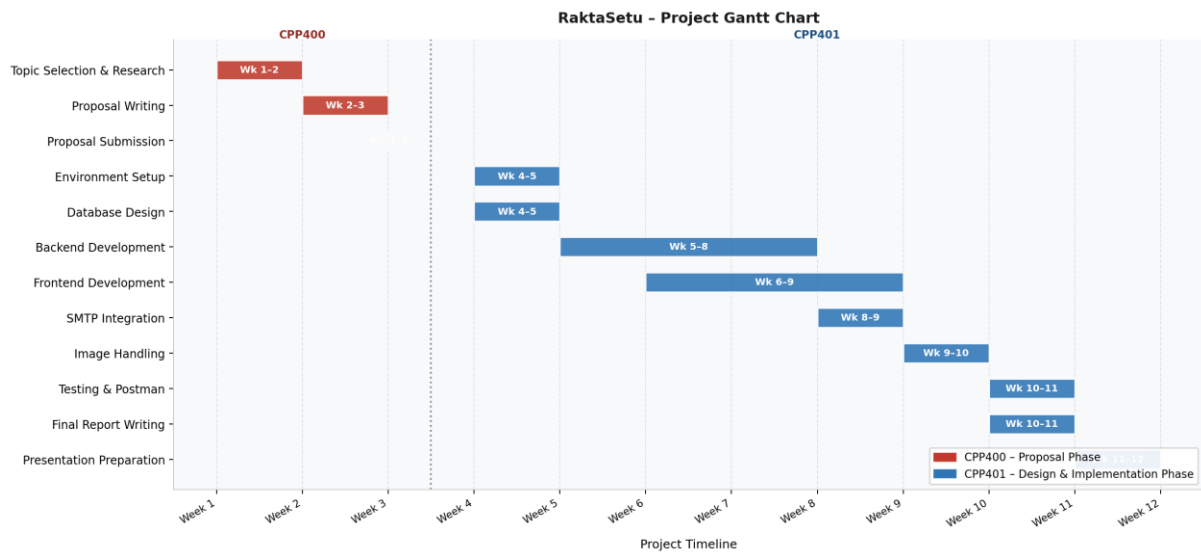


Figure 6-1: Project Gantt Chart

CHAPTER 7: CONCLUSION, LIMITATIONS AND FUTURE SCOPE

7.1 Conclusion

BloodBuddy set out to close five evidence-backed gaps — unstructured social-media requests, no searchable donor registry, manual hospital records, no automated alerting, and no moderation — with a structured, role-based web platform, and the frontend through which users experience all five closures is complete and verified: 266/266 functional checks, 92/92 responsive page×width combinations, and 16/16 real-HTTP transport checks. The full request lifecycle — submission, moderation, hospital routing, compatibility-correct donor matching, fulfilment with stock decrement — works end-to-end across all four roles plus the guest emergency flow.

The project's most instructive results are methodological. Centralising the compatibility chart in one module exposed how easily a clinical rule drifts when duplicated — and how the matching direction itself is the safety-critical detail. The API facade design made the mock-to-real backend swap a one-flag change and, just as importantly, made the transport layer testable: the one defect that would have silently broken every real call was invisible to 261 functional checks and was caught only by exercising the fetch path over HTTP, then pinned by a negative-tested guard. The project traces cleanly from documented field evidence through requirements, layered design, phased implementation, and evidence-based verification — a complete software development lifecycle in miniature.

7.2 Limitations

- The demonstration build persists data in the browser; the Spring MVC + MySQL backend that makes the system durable and multi-user is the scheduled implementation phase, and the browser store is explicitly not a backend.
- Matching, authorisation, and status rules are enforced client-side in the demonstration; the authoritative service-layer enforcement specified in Section 3.9 must land with the backend or the security model is advisory.
- Hospital and inventory data is seeded demonstration data; no partner hospital is connected, and the local hospital APIs are designed and stubbed but not live.
- Email notifications are designed (EMAIL_NOTIFICATION entity, SMTP configuration via environment variables) but not yet sending, as delivery depends on the backend phase.
- Concurrency is untested beyond the automated suites' scope; server-side guarantees (e.g., a database-level uniqueness on request matching) are specified but not yet in force.
- The requirement base derives from the proposal's document-based analysis; broader field validation with hospitals and donor communities remains future work.

7.3 Future Scope

- Complete the Spring MVC backend: six Hibernate entities with 1:1/1:N/M:N relationships, Spring Security RBAC, server-side validation, and the hospital-scoped API modules behind the existing facade contract.
- SMTP registration confirmation and urgent-request donor alerts, with credentials via environment variables; optional login alerts and password-reset emails.
- Onboard the five partner hospitals onto the local APIs for real-time inventory sharing; add cross-hospital stock search so requesters can locate scarce groups region-wide.
- SMS push alongside email for urgent requests, given Nepal's mobile-first usage patterns.

- Donor engagement: eligibility intervals, donation history verification, reminders, and recognition badges to sustain the voluntary donor base.
- Deployment to a cloud host with HTTPS, database backups, and monitoring; multilingual (Nepali) interface; and a native mobile companion if usage data justifies it.

REFERENCES

- Bauer, C., & King, G. (2015). Java persistence with Hibernate (2nd ed.). Manning Publications.
- Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Doctoral dissertation, University of California, Irvine]. University of California.
- Hibernate ORM Documentation. (2024). Hibernate getting started guide. https://docs.jboss.org/hibernate/orm/current/quickstart/html_single/
- New Business Age. (2025). Nepal's digital crossroads: Growth vs barriers in the race to transform. <https://www.newbusinessage.com/news/43801>
- Nepal Economic Forum. (n.d.). Barriers to local digital innovation in Nepal. <https://nepaleconomicforum.org/barriers-to-local-digital-innovation-in-nepal/>
- NIPoRe. (2023). Digitalization of Nepal — few policies and possible challenges. <https://nipore.org/digitalization-of-nepal-few-policies-and-possible-challenges/>
- Sandhu, R. S., & Samarati, P. (1994). Access control: Principles and practice. IEEE Communications Magazine, 32(9), 40–48.
- Spring Framework Documentation. (2024). Spring MVC — Web on Servlet Stack. <https://docs.spring.io/spring-framework/reference/web/webmvc.html>
- World Bank. (2025). Unlocking Nepal's growth potential: Nepal economic memorandum. Washington, D.C.: The World Bank Group.
- World Health Organisation. (2023). Blood safety and availability. WHO Fact Sheets. <https://www.who.int/news-room/fact-sheets/detail/blood-safety-and-availability>

APPENDICES

Appendix A: Blood Group Compatibility Chart

The compatibility chart below is the authoritative reference implemented in `bb-compat.js` and used by the donor matching algorithm to suggest compatible donor blood groups when an exact match is unavailable. The chart is written recipient → compatible donor groups; matching code must read it in that direction.

Table A-1: Recipient → Compatible Donor Blood Groups

Recipient Blood Group	Compatible Donor Blood Groups
A+	A+, A-, O+, O-
A-	A-, O-
B+	B+, B-, O+, O-
B-	B-, O-
AB+	All blood groups (Universal Recipient)
AB-	AB-, A-, B-, O-
O+	O+, O-
O-	O- (Universal Donor)

Appendix B: REST API Endpoint Reference

Full listing of every endpoint the frontend facade consumes (mirroring the proposal's Appendix B), with the hospital-scoped local APIs for the five partner hospitals.

Table B-1: REST API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register a new user (donor, requester, hospital staff, or admin)
POST	/api/auth/login	Authenticate user and return session token
GET	/api/donors/search	Search donors by blood group and location (paginated)
GET	/api/donors/{id}	Retrieve a specific donor's profile
GET	/api/requests	List/filter requests (?requester=&donor=&hospital=&guest=)
POST	/api/requests	Submit a new blood request
GET	/api/requests/{id}	Retrieve details of a specific blood request
PATCH	/api/requests/{id}	Update request status (accept / fulfil / cancel / moderate / forward)
GET	/api/hospitals/inventory	Retrieve blood bank inventory for a hospital
PUT	/api/hospitals/inventory/{bloodGroup}	Update blood bank stock levels
GET	/api/admin/users	Admin: retrieve all registered users
PATCH	/api/admin/users/{id}	Admin: set user status (activate/suspend)
GET	/api/hospitals/{hospitalId}/inventory	Local: full blood inventory for a specific

		partner hospital (TUTH, Bir, Patan, Manmohan, NMCTH)
PUT	/api/hospitals/{hospitalId}/inventory/{bloodGroup}	Local: update stock level for a specific blood group at a partner hospital
POST	/api/hospitals/{hospitalId}/requests	Local: submit a blood request directly to a partner hospital's queue
GET	/api/hospitals/{hospitalId}/requests/{requestId}/status	Local: check the fulfilment status of a blood request at a partner hospital
GET	/api/hospitals/{hospitalId}/donors/nearby	Local: registered donors near a partner hospital, filtered by blood group
GET / PUT	/api/profile/{role}	Retrieve / save the signed-in user's role profile (incl. Base64 photograph)

Appendix C: Database Schema

To be exported from MySQL Workbench during the backend phase: table definitions for USER, DONOR, BLOOD_REQUEST, HOSPITAL, BLOOD_INVENTORY and EMAIL_NOTIFICATION with FK constraints, the role/status enums, and the hospital_id + blood_group uniqueness on inventory rows.

Appendix D: Selected Source Code

Selected implementation excerpts (full files in the submitted codebase):

- CODE/js/bb-compat.js — the Appendix A compatibility chart and direction-correct matching helpers.
- CODE/js/bb-api.js — the API facade: endpoint table, mock adapter, real fetch transport, error normalisation, and the documented BASE invariant.
- CODE/js/nav.js — session model, ROLE_LINKS/ROLE_CTA tables, RBAC guard map, and return-URL continuation logic.
- CODE/js/bb-validate.js — shared validation rules and inline error rendering.
- CODE/js/bb-paginate.js — reusable pager (page sizes, prev/next, auto-hide).

Appendix E: QA Evidence and Reproduction Steps

The three verification tools and their recorded results:

- resptest/qa-harness.html — 266/266 checks (functional, all four roles, guest flow, transport guards). Run: serve the repo root, then headless Chrome with a fresh profile and virtual time budget; the harness prints a per-check PASS/FAIL report.
- resptest/responsive-check.html — 92/92 page×width combinations (23 pages at 360/430/768/1280) with no horizontal overflow.
- resptest/real-mode-check.html + tools/mock_api_server.py — 16/16 over real HTTP; the stub's stdout logs each GET /api/... → 200, and the store is pinned empty so rendered rows can only have arrived over HTTP.

- `resptest/visual-forms.html` — computed-style validation-error probe (6.03:1 contrast, error ring cascade) with saved screenshots.

[INSERT SCREENSHOT: QA harness report; responsive matrix; real-mode stub log — see Figure 5-1]

Appendix F: Additional Screenshots

Additional interface evidence beyond those placed in-chapter (landing, about, contact, legal pages, all four role portals, and the branded 404):

[INSERT SCREENSHOT: `landing.html` — hero and quick search]

[INSERT SCREENSHOT: `auth-login.html` / `auth-register.html`]

[INSERT SCREENSHOT: `donor-dashboard.html` — nearby compatible requests with accept/decline]

[INSERT SCREENSHOT: `hospital-requests.html` — queue with Donor matched / Fulfilled states]

[INSERT SCREENSHOT: `terms.html` / `privacy.html` / `404.html`]